

1. Write a program in C to find the second smallest element in an array.

```
#include<stdio.h>
#include<limits.h>

int main(){
    int n;
    printf("enter the number of array elements to be inserted:");
    scanf("%d",&n);
    if(n<2){
        printf("enter more than one elements in the array");
    }
    int arr[n];
    printf("Enter %d elements:",n);
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int smallest=INT_MAX;
    int SecondSmallest=INT_MAX;
    for(int i=0;i<n;i++){
        if(arr[i]<smallest){
            SecondSmallest=smallest;
            smallest=arr[i];
        }
        else if(arr[i]>smallest && arr[i]<SecondSmallest){
            SecondSmallest=arr[i];
        }
    }
    if(SecondSmallest==INT_MAX){
        printf("there is no second smallest element");
    }
}
```

```

}
else{
    printf("the second smallest element:%d",SecondSmallest);
}
return 0;
}

```

Output:

```

enter the number of array elements to be inserted:4
Enter 4 elements:10 20 30 40
the second smallest element:20

```

2. Write a program in C to find the sum of the left diagonals of matrix.

```

#include<stdio.h>

int main(){
    int n, i, j;
    int matrix [10][10];
    int sum=0;

    printf("Enter the order of the matrix:",n);
    scanf("%d",&n);

    printf("enter matrix elements:\n");
    for (i=0; i<n; i++){
        for(j=0; j<n; j++){
            scanf("%d",&matrix[i][j]);
        }
    }
}

```

```

}
for (i=0; i<n; i++){
    sum=sum+matrix[i][i];
}
printf("the sum of diagonal matrix:%d",sum);
return 0;
}

```

Output:

```

Enter the order of the matrix:2
enter matrix elements:
10 05 45 17
the sum of diagonal matrix:27

```

3. Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

A. FCFS

```

#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], wt[n], tat[n], rt[n], ct[n];

    for(i = 0; i < n; i++) {

```

```
printf("\nProcess %d\n", i+1);
printf("Arrival Time: ");
scanf("%d", &at[i]);
printf("Burst Time: ");
scanf("%d", &bt[i]);
}
```

```
ct[0] = at[0] + bt[0];
tat[0] = ct[0] - at[0];
wt[0] = tat[0] - bt[0];
rt[0] = wt[0];
```

```
for(i = 1; i < n; i++) {
    if(ct[i-1] < at[i])
        ct[i] = at[i] + bt[i];
    else
        ct[i] = ct[i-1] + bt[i];
```

```
tat[i] = ct[i] - at[i];
wt[i] = tat[i] - bt[i];
rt[i] = wt[i];
}
```

```
printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\tRT\n");
```

```

float total_wt = 0, total_tat = 0, total_rt = 0;
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           i+1, at[i], bt[i], ct[i], wt[i], tat[i], rt[i]);

    total_wt += wt[i];
    total_tat += tat[i];
    total_rt += rt[i];
}

printf("\nAverage Waiting Time = %.2f", total_wt/n);
printf("\nAverage Turnaround Time = %.2f", total_tat/n);
printf("\nAverage Response Time = %.2f\n", total_rt/n);
return 0;
}

```

Output:

```

Process 1
Arrival Time: 0
Burst Time: 2

Process 2
Arrival Time: 0
Burst Time: 4

Process 3
Arrival Time: 0
Burst Time: 6

Process 4
Arrival Time: 0
Burst Time: 8

Process AT    BT    CT    WT    TAT    RT
P1      0      2      2      0      2      0
P2      0      4      6      2      6      2
P3      0      6     12      6     12      6
P4      0      8     20     12     20     12

Average Waiting Time = 5.00
Average Turnaround Time = 10.00
Average Response Time = 5.00

```

